

1. Проектирование сервиса ставок

1.1 Границы сервиса ставок (bidding Service)

Bidding Service (ранее – часть монолита, в новой архитектуре – отдельный микросервис) является входной точкой для всех RTB-запросов от DSP и Publishers. Его основная задача – координировать процесс аукциона и формировать финальный ответ, но саму логику аукциона он делегирует выделенному Auction Engine.

Что входит в границы Bidding Service

- Приём Bid Request от внешних систем (DSP, SSP) по протоколу OpenRTB (HTTP/2 + JSON) через API Gateway.
- Валидация запроса (обязательные поля, формат, подписи).
- Идентификация пользователя – извлечение user_id из cookie, device ID, IP и т.д.
- Получение пользовательского профиля – запрос к User Profile Service (или чтение из Redis-кеша) для получения сегментов аудитории, frequency caps.
- Поиск подходящих кампаний – запрос к Сервису рекламных кампаний (или чтение из Redis-кеша) для получения всех активных кампаний, релевантных для данного запроса (по гео, устройству, времени, аудиторным сегментам).
- Сбор кандидатов – формирование списка участников аукциона для Auction Engine.
- Вызов Сервису рекламных кампаний (синхронно, gRPC) для определения победителя.
- Запрос баннера – при получении победителя вызывается Creative Service (синхронно, gRPC) для генерации финальной разметки.
- Формирование и отправка Bid Response обратно клиенту.
- Асинхронная публикация событий (клики/показы) в Kafka для дальнейшей обработки.

Что НЕ входит в границы сервиса ставок:

- Логика самого аукциона (правила, взвешивание) – это ответственность Auction Engine.
- Генерация баннера – Creative Service.
- Управление кампаниями и их правилами – Campaign Rules Service (только чтение через кеш/CDC).

- Долгосрочное хранение данных – сервис stateless, все данные получает из кешей и внешних сервисов.

1.2 API сервиса

- Внешний API (через API Gateway)
- Протокол: gRPC поверх HTTP/2 (для низкой задержки и бинарной сериализации).
- Формат сообщений: Protocol Buffers.
- Основные API (ориентировочный вариант)
 - BidRequest → BidResponse (основной RTB-эндпоинт)
Назначение: аукцион за показ.
 - Входящие поля: ad_unit_id, user_id (или cookie/device_id), site/app, geo, device_type, context, timestamp.
 - Исходящие поля: winner_campaign_id, creative_url, tracking_pixels, price_cpm, ttl_ms, debug_info (для отладки).
 - HealthCheck → HealthStatus Для Kubernetes readiness/liveness. Возвращает статус Redis, PostgreSQL replica, Kafka producer.
 - Экспорт внутренних метрик (latency, error rates, cache hit/miss) для систем мониторинга (Prometheus.)

1.3 Зависимости от других сервисов

Зависимость	Тип связи	Протокол	Цель
Сервис рекламных кампаний	Синхронная критический вызов	gRPC	Получение правил, лимитов, статусов кампаний.
Сервис выдачи банера	Синхронная критический вызов	gRPC	Генерация баннера для победителя.
Redis	Синхронно	RESP	Горячие данные: правила, лимиты, профили пользователей, счётчики частоты показов.
PostgreSQL (read-replica)	Синхронно, только SELECT по ключам	PG protocol	Источник истины для правил и лимитов, если нет в кэше.

Kafka	Асинхронно (fire-and-forget)	Kafka protocol	Публикация событий events.raw (показ, клик, viewability).
API Gateway	Синхронно	gRPC/HTTP	Маршрутизация, rate limiting, auth, трейсинг.

Критический путь (должен укладываться в <80 мс) включает:

- чтение из Redis;
- вызов сервис рекламных кампаний;
- вызов сервис выдачи.

Важные паттерны для зависимостей:

- Cache-aside: сначала Redis, если нет — PostgreSQL, затем обновление кэша.
- Circuit Breaker: для защиты от каскадных отказов при проблемах с Kafka или PostgreSQL.
- Идемпотентность: события в Kafka должны быть идемпотентными, чтобы повторная отправка не ломала учёт.

Модель данных сервиса ставок

У сервиса ставок нет собственной транзакционной БД. Он использует:

- Redis — для горячих данных (в виде структур: hashes, sorted sets, bitmaps).
- PostgreSQL read-replica — как источник истины (только SELECT).